

Exercise 1: Inventory Management System

Scenario

You are developing an inventory management system for a warehouse. Efficient data storage and retrieval are crucial.

Objective

Develop an Inventory Management System using Java that allows users to add, update, and delete product details efficiently using a suitable data structure.

Explanation

Data structures and algorithms play an important role in inventory management because they help organize, store, and retrieve product information efficiently. For this exercise, a **HashMap** is used since it provides fast insertion, searching, updating, and deletion of products using the Product ID as the key. This makes it suitable for handling large inventories.

Code

Product.java

```
import java.util.*;
```

```
class Product {
```

```
    private int productId;
```

```
    private String productName;
```

```
    private int quantity;
```

```
    private double price;
```

```
    public Product(int productId, String productName, int quantity, double price) {
```

```
        this.productId = productId;
```

```
        this.productName = productName;
```

```
        this.quantity = quantity;
```

```
        this.price = price;
```

```
    }
```

```
public int getProductId() {  
    return productId;  
}
```

```
public void setQuantity(int quantity) {  
    this.quantity = quantity;  
}
```

```
public void setPrice(double price) {  
    this.price = price;  
}
```

@Override

```
public String toString() {  
    return "Product ID : " + productId +  
        "\nProduct Name : " + productName +  
        "\nQuantity : " + quantity +  
        "\nPrice : " + price + "\n";  
}  
}
```

InventoryManagement.java

```
import java.util.HashMap;
```

```
class InventoryManagement {
```

```
    HashMap<Integer, Product> inventory = new HashMap<>();
```

```
    public void addProduct(Product product) {  
        inventory.put(product.getProductId(), product);
```

```
        System.out.println("Product Added Successfully.");
    }

    public void updateProduct(int id, int quantity, double price) {

        if (inventory.containsKey(id)) {

            Product product = inventory.get(id);

            product.setQuantity(quantity);
            product.setPrice(price);

            System.out.println("Product Updated Successfully.");

        } else {

            System.out.println("Product Not Found.");

        }
    }

    public void deleteProduct(int id) {

        if (inventory.containsKey(id)) {

            inventory.remove(id);

            System.out.println("Product Deleted Successfully.");
```

```

    } else {

        System.out.println("Product Not Found.");

    }

}

public void displayProducts() {

    for (Product product : inventory.values()) {

        System.out.println(product);

    }

}

}

```

Main.java

```

public class Main {

    public static void main(String[] args) {

        InventoryManagement inventory = new InventoryManagement();

        inventory.addProduct(new Product(101, "Laptop", 10, 55000));

        inventory.addProduct(new Product(102, "Mouse", 50, 500));
    }
}

```

```
inventory.updateProduct(101, 15, 53000);
```

```
inventory.deleteProduct(102);
```

```
System.out.println("\nAvailable Products\n");
```

```
inventory.displayProducts();
```

```
}
```

```
}
```

Output

Product Added Successfully.

Product Added Successfully.

Product Updated Successfully.

Product Deleted Successfully.

Available Products

Product ID : 101

Product Name : Laptop

Quantity : 15

Price : 53000.0

The screenshot shows an IDE with the following components:

- EXPLORER:** Shows a project structure with folders `.vscode`, `InventorySystem`, and files `InventoryManagement.java`, `Main.java`, and `Product.java`.
- EDITOR:** Displays `Main.java` with the following code:

```
1 package InventorySystem;
2 public class Main {
3
4     public static void main(String[] args) {
5
6         InventoryManagement inventory = new InventoryManagement();
7
8         inventory.addProduct(new Product(productId: 101, productName: "Laptop", quantity: 10, price: 55000));
9
10        inventory.addProduct(new Product(productId: 102, productName: "Mouse", quantity: 50, price: 500));
11
12        inventory.updateProduct(id: 101, quantity: 15, price: 53000);
13
14        inventory.deleteProduct(id: 102);
15
16        System.out.println(x: "\nAvailable Products\n");
17
18        inventory.displayProducts();
19
20    }
21
22 }
```
- TERMINAL:** Shows the command to run the program and its output:

```
PS C:\Users\User\Downloads\OUTPUTS> & "C:\Program Files\Eclipse Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe" "-Xgc:showCodeDetails\n ExceptionMessages" "-cp" "C:\Users\User\AppData\Roaming\Code\User\workspaceStorage\def5f49e68a57468a7a3467e16796f3\redhat_java\jdt_ws\OUTPUTS_2976b6c9\bin" "InventorySystem.Main"
Product Added Successfully.
Product Added Successfully.
Product Updated Successfully.
Product Deleted Successfully.

Available Products

Product ID : 101
Product Name : Laptop
Quantity : 15
Price : 53000.0
```
- RIGHT SIDEBAR:** Includes a 'SESSIONS' panel and a 'Run Main' button.

Time Complexity

Operation	Complexity
-----------	------------

Add Product	$O(1)$
-------------	--------

Update Product	$O(1)$
----------------	--------

Delete Product	$O(1)$
----------------	--------

Search Product	$O(1)$
----------------	--------

Conclusion

Successfully implemented an Inventory Management System using Java and HashMap. The system efficiently performs add, update, delete, and display operations, making it suitable for managing large inventories.

Exercise 2: E-commerce Platform Search Function

Scenario

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Objective

The objective of this exercise is to implement and compare **Linear Search** and **Binary Search** algorithms for searching products in an e-commerce platform. The performance of both algorithms is analyzed using Big O notation.

Explanation

Big O notation is used to measure the efficiency of an algorithm based on the size of the input. Linear Search checks each element one by one, making it suitable for small or unsorted datasets. Binary Search works only on sorted data and repeatedly divides the search space into two halves, making it much faster for large datasets.

Best, Average and Worst Case

- **Best Case:** The product is found at the first position.
 - **Average Case:** The product is found somewhere in the middle.
 - **Worst Case:** The product is found at the last position or is not present.
-

Code

Product.java

```
class Product {

    int productId;

    String productName;

    String category;

    public Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    @Override
    public String toString() {
        return productId + " " + productName + " " + category;
    }
}
```

```
}
```

SearchAlgorithms.java

```
import java.util.Arrays;
```

```
import java.util.Comparator;
```

```
class SearchAlgorithms {
```

```
    // Linear Search
```

```
    public static Product linearSearch(Product[] products, int id) {
```

```
        for (Product product : products) {
```

```
            if (product.productId == id) {
```

```
                return product;
```

```
            }
```

```
        }
```

```
        return null;
```

```
    }
```

```
    // Binary Search
```

```
    public static Product binarySearch(Product[] products, int id) {
```

```
        Arrays.sort(products, Comparator.comparingInt(p -> p.productId));
```

```
        int left = 0;
```

```
        int right = products.length - 1;
```

```
        while (left <= right) {
```



```
int mid = (left + right) / 2;

if (products[mid].productId == id) {
    return products[mid];
}

if (products[mid].productId < id) {
    left = mid + 1;
} else {
    right = mid - 1;
}

}

return null;
}

}
```

Main.java

```
public class Main {

    public static void main(String[] args) {

        Product[] products = {

            new Product(103, "Keyboard", "Electronics"),
            new Product(101, "Laptop", "Electronics"),
            new Product(104, "Headphones", "Accessories"),
            new Product(102, "Mouse", "Electronics")
        }
    }
}
```

```
};

System.out.println("Linear Search");

Product p1 = SearchAlgorithms.linearSearch(products, 104);

if (p1 != null)
    System.out.println(p1);

System.out.println();

System.out.println("Binary Search");

Product p2 = SearchAlgorithms.binarySearch(products, 104);

if (p2 != null)
    System.out.println(p2);

}

}
```

Output

Linear Search

104 Headphones Accessories

Binary Search

104 Headphones Accessories

```
1 package SearchSystem;
2
3 public class Main {
4     public static void main(String[] args)
5     {
6         Product[] products = { new Product(productId: 103, productName: "Keyboard", category: "Electronics"), new Product(productId: 104, productName: "Headphones", category: "Accessories") };
7         System.out.println(x: "Linear Search");
8         Product p1 = SearchAlgorithms.linearSearch(products, id: 104);
9         if (p1 != null)
10             System.out.println(p1);
11         System.out.println();
12         System.out.println(x: "Binary Search");
13         Product p2 = SearchAlgorithms.binarySearch(products, id: 104);
14         if (p2 != null)
15             System.out.println(p2);
16     }
17 }
```

PS C:\Users\User\Downloads\OUTPUTS> c:\cd 'c:\Users\User\Downloads\OUTPUTS'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe' -Xmx1024m -Xms64m -cp 'C:\Users\User\AppData\Roaming\Code\User\workspaceStorage\def5fed9e08a574e0a7a367e16796f3\redhat.java\jdk_ws\OUTPUTS_2976bb09\bin' 'InventorySystem.Main'

Product ID : 104
Product Name : Laptop
Quantity : 15
Price : 53000.0

Linear Search
104 Headphones Accessories

Binary Search
104 Headphones Accessories

Time Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

Conclusion

Successfully implemented Linear Search and Binary Search algorithms in Java. Compared their performance using Big O notation and observed that Binary Search is more efficient for searching products in sorted datasets, making it more suitable for large e-commerce platforms.